

42. Build a document-based AI assistant that supports multiple uploaded documents and question

Code:

```
from sentence_transformers import SentenceTransformer
from transformers import pipeline
import faiss
import numpy as np
```

```
# =====
```

```
# MULTI-DOCUMENT AI ASSISTANT
```

```
# =====
```

```
print("=====")
```

```
print("    MULTI-DOCUMENT AI ASSISTANT")
```

```
print("=====")
```

```
# =====
```

```
# 1. MULTIPLE DOCUMENTS
```

```
# =====
```

```
documents = {
```

```
    "AI.txt": """
```

```
    Artificial Intelligence is a branch of computer science
    that focuses on creating intelligent systems.
```

```
    AI can be used in healthcare, education, finance and
    many other fields.
```

```

"""
,

"ML.txt": """
Machine Learning is a subset of Artificial Intelligence.
It allows computers to learn patterns from data and
make predictions without being explicitly programmed.
"""
,

"NLP.txt": """
Natural Language Processing is a branch of Artificial
Intelligence that helps computers understand, process
and generate human language.
"""
,

"DL.txt": """
Deep Learning is a type of Machine Learning that uses
artificial neural networks with multiple layers.
It is commonly used in image recognition, speech
recognition and other complex tasks.
"""

}

print("\nDocuments loaded:")
for name in documents:
    print("-", name)

# =====

```

```
# 2. CREATE DOCUMENT CHUNKS
```

```
# =====
```

```
chunks = []
```

```
source_names = []
```

```
for name, content in documents.items():
```

```
    parts = [
```

```
        part.strip()
```

```
        for part in content.split("\n")
```

```
        if part.strip()
```

```
    ]
```

```
    for part in parts:
```

```
        chunks.append(part)
```

```
        source_names.append(name)
```

```
print("\nTotal document chunks:", len(chunks))
```

```
# =====
```

```
# 3. LOAD EMBEDDING MODEL
```

```
# =====
```

```
print("\nLoading embedding model...")
```

```
embedding_model = SentenceTransformer(
```

```
"all-MiniLM-L6-v2"
)

print("Embedding model loaded.")

# =====
# 4. CREATE EMBEDDINGS
# =====

print("\nCreating document embeddings...")

embeddings = embedding_model.encode(
    chunks
)

embeddings = np.array(
    embeddings
).astype("float32")

print("Embeddings created successfully.")

# =====
# 5. CREATE FAISS VECTOR DATABASE
# =====

print("\nCreating FAISS vector database...")
```

```
dimension = embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(embeddings)
```

```
print("FAISS database created.")
```

```
print("Total vectors:", index.ntotal)
```

```
# =====
```

```
# 6. LOAD LANGUAGE MODEL
```

```
# =====
```

```
print("\nLoading language model...")
```

```
llm = pipeline(  
    "text2text-generation",  
    model="google/flan-t5-small"  
)
```

```
print("Language model loaded successfully.")
```

```
# =====
```

```
# 7. START AI ASSISTANT
```

```
# =====
```

```
print("\n=====")
```

```
print("          AI ASSISTANT READY")
print("=====")
```

```
print("You can ask questions about all documents.")
print("Type 'exit' to stop.")
```

```
while True:
```

```
    # -----
    # Get question
    # -----
```

```
    question = input("\nYou: ")
```

```
    # -----
    # Exit
    # -----
```

```
    if question.lower() == "exit":
```

```
        print("\nAssistant: Goodbye!")
        break
```

```
    # =====
    # 8. CREATE QUERY EMBEDDING
    # =====
```

```
query_embedding = embedding_model.encode(  
    [question]  
)
```

```
query_embedding = np.array(  
    query_embedding  
)  
.astype("float32")
```

```
# =====  
# 9. SEARCH MULTIPLE DOCUMENTS  
# =====
```

```
k = 3
```

```
distances, indices = index.search(  
    query_embedding,  
    k  
)
```

```
# =====  
# 10. RETRIEVE RELEVANT INFORMATION  
# =====
```

```
context = ""
```

```
selected_sources = []
```

```
for i in indices[0]:
```

```
    context += chunks[i] + "\n"
```

```
    if source_names[i] not in selected_sources:
```

```
        selected_sources.append(
```

```
            source_names[i]
```

```
        )
```

```
# =====
```

```
# 11. CREATE PROMPT
```

```
# =====
```

```
prompt = ""
```

You are a document-based AI assistant.

Answer the question using only the information
provided in the document context.

If the answer is not available, say:

"I could not find the answer in the documents."

Document Context:

```
{
```

Question:

```
{
```

Answer:

```
"".format(  
    context,  
    question  
)
```

```
# =====  
# 12. GENERATE ANSWER  
# =====
```

```
result = llm(  
    prompt,  
    max_new_tokens=120  
)
```

```
answer = result[0]["generated_text"]
```

```
# =====  
# 13. DISPLAY ANSWER  
# =====
```

```
print("\nAssistant:", answer)
```

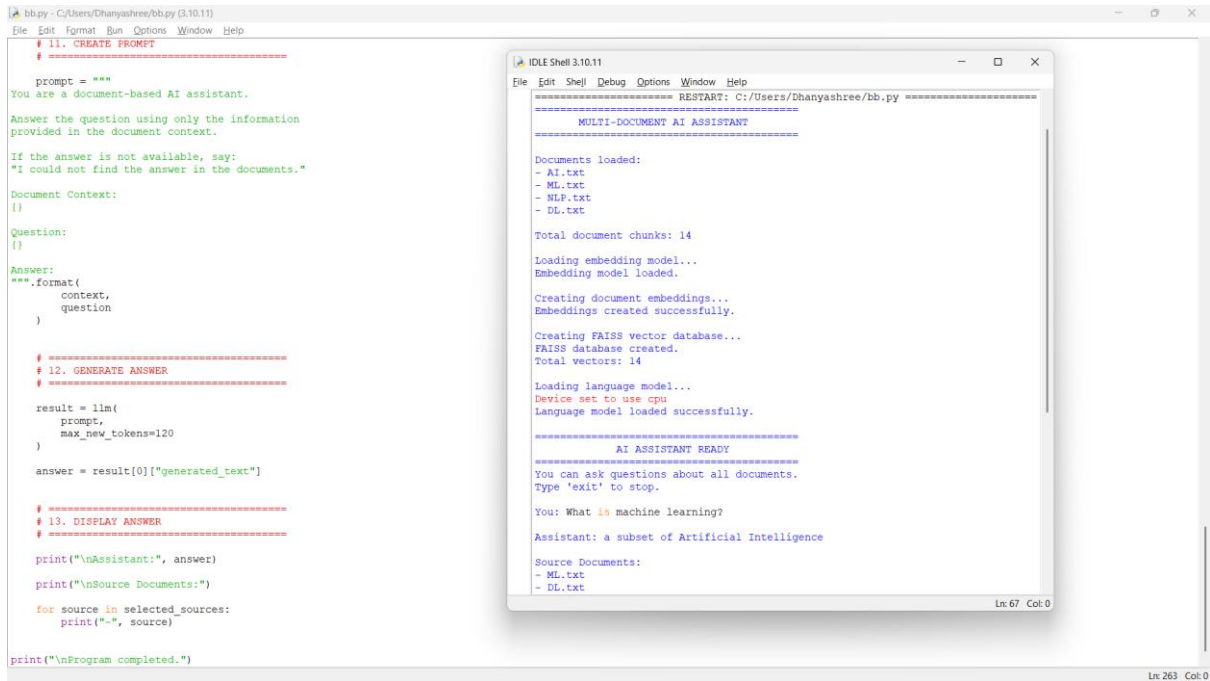
```
print("\nSource Documents:")
```

```
for source in selected_sources:
```

```
print("-", source)
```

```
print("\nProgram completed.")
```

output:



```
bb.py - C:/Users/Dhanyashree/bb.py (3.10.11)
File Edit Format Run Options Window Help

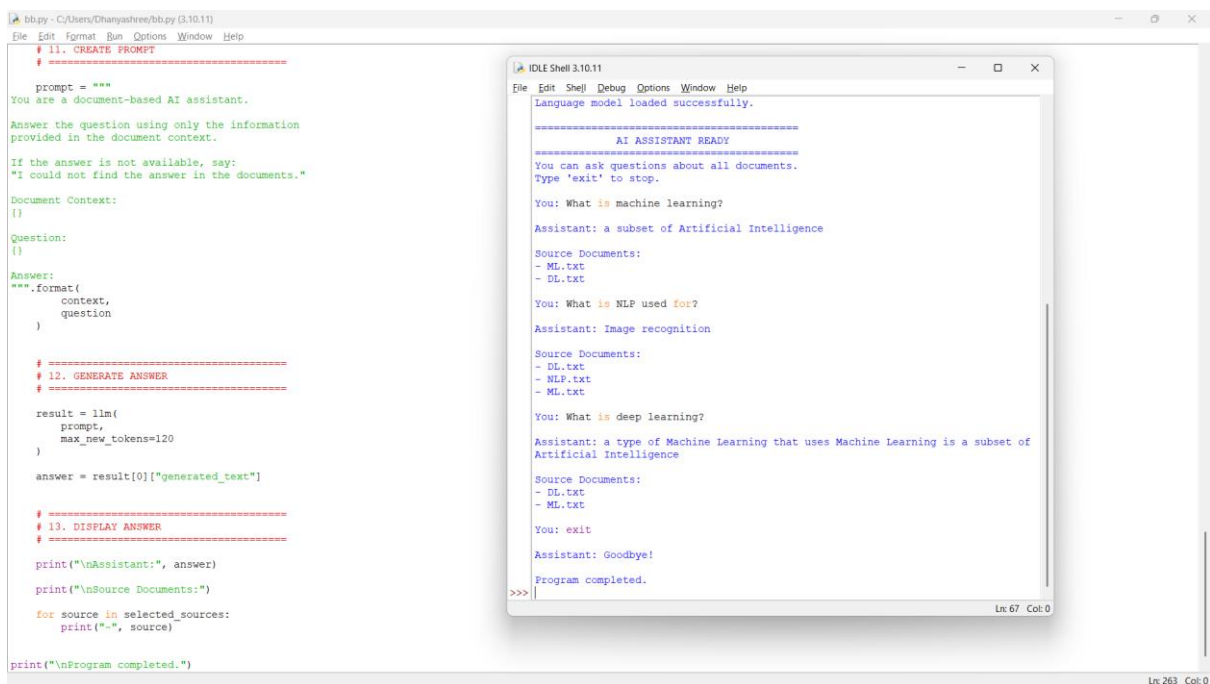
# 11. CREATE PROMPT
# =====
prompt = """
You are a document-based AI assistant.
Answer the question using only the information
provided in the document context.
If the answer is not available, say:
"I could not find the answer in the documents."
Document Context:
{}
Question:
{}
Answer:
"""
def format(context, question):
    return prompt.format(context, question)

# 12. GENERATE ANSWER
# =====
result = llm(
    prompt=format(context, question),
    max_new_tokens=120
)
answer = result[0]["generated_text"]

# 13. DISPLAY ANSWER
# =====
print("\nAssistant:", answer)
print("\nSource Documents:")
for source in selected_sources:
    print("-", source)

print("\nProgram completed.")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
RESTART: C:/Users/Dhanyashree/bb.py
=====
MULTI-DOCUMENT AI ASSISTANT
=====
Documents loaded:
- AI.txt
- ML.txt
- NLP.txt
- DL.txt
Total document chunks: 14
Loading embedding model...
Embedding model loaded.
Creating document embeddings...
Embeddings created successfully.
Creating FAISS vector database...
FAISS database created.
Total vectors: 14
Loading language model...
Device set to use cpu
Language model loaded successfully.
=====
AI ASSISTANT READY
=====
You can ask questions about all documents.
Type 'exit' to stop.
You: What is machine learning?
Assistant: a subset of Artificial Intelligence
Source Documents:
- ML.txt
- DL.txt
Ln 67 Col 0
```



```
bb.py - C:/Users/Dhanyashree/bb.py (3.10.11)
File Edit Format Run Options Window Help

# 11. CREATE PROMPT
# =====
prompt = """
You are a document-based AI assistant.
Answer the question using only the information
provided in the document context.
If the answer is not available, say:
"I could not find the answer in the documents."
Document Context:
{}
Question:
{}
Answer:
"""
def format(context, question):
    return prompt.format(context, question)

# 12. GENERATE ANSWER
# =====
result = llm(
    prompt=format(context, question),
    max_new_tokens=120
)
answer = result[0]["generated_text"]

# 13. DISPLAY ANSWER
# =====
print("\nAssistant:", answer)
print("\nSource Documents:")
for source in selected_sources:
    print("-", source)

print("\nProgram completed.")

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Language model loaded successfully.
=====
AI ASSISTANT READY
=====
You can ask questions about all documents.
Type 'exit' to stop.
You: What is machine learning?
Assistant: a subset of Artificial Intelligence
Source Documents:
- ML.txt
- DL.txt
You: What is NLP used for?
Assistant: Image recognition
Source Documents:
- DL.txt
- NLP.txt
- ML.txt
You: What is deep learning?
Assistant: a type of Machine Learning that uses Machine Learning is a subset of
Artificial Intelligence
Source Documents:
- DL.txt
- ML.txt
You: exit
Assistant: Goodbye!
Program completed.
>>>
Ln 67 Col 0
```